

Machine Learning: Comprehensive Study Guide

Based on CS181 Course Materials

May 10, 2025

Contents

1	Introduction	4
2	Regression	4
2.1	Linear Regression Basics	4
2.1.1	The Bias Trick	4
2.1.2	Least Squares Loss Function	4
2.1.3	Derivation of Optimal Weights	4
2.2	Basis Functions	5
2.3	Regularization	5
2.3.1	Ridge Regression (L2 Regularization)	5
2.3.2	Lasso Regression (L1 Regularization)	5
2.3.3	Elastic Net Regularization	6
2.4	Bias-Variance Tradeoff	6
2.5	Bayesian Linear Regression	6
2.5.1	Posterior Predictive Distribution	6
2.5.2	Marginal Likelihood	7
3	Classification	7
3.1	Loss Functions for Classification	7
3.1.1	0/1 Loss	7
3.1.2	Hinge Loss	7
3.1.3	Logistic Loss	7
3.2	Gradient Descent	7
3.2.1	Batch vs. Stochastic Gradient Descent	8
3.3	Perceptron Algorithm	8
3.4	Logistic Regression	8
3.4.1	Multi-class Logistic Regression	8
3.5	Generative vs. Discriminative Models	9
3.5.1	Generative Models for Classification	9
3.5.2	Naive Bayes	9
3.5.3	Comparing Discriminative and Generative Models	9
4	Neural Networks & Model Selection	10
4.1	Neural Network Basics	10
4.1.1	Activation Functions	10
4.1.2	Forward Propagation	10
4.2	Backpropagation	10
4.3	Neural Networks for Regression and Classification	11
4.3.1	Regression with Neural Networks	11
4.3.2	Classification with Neural Networks	11
4.4	Model Selection and Regularization	12

4.4.1	Cross-Validation	12
4.4.2	Regularization in Neural Networks	12
5	Support Vector Machines (SVMs)	12
5.1	Maximum Margin Classification	12
5.2	Hard-Margin SVM	12
5.3	Soft-Margin SVM	13
5.4	Hinge Loss in SVMs	13
5.5	Kernel Trick	13
5.6	Support Vectors	13
5.7	SVM vs Logistic Regression	14
6	Clustering	14
6.1	K-means Clustering	14
6.1.1	K-means Objective	14
6.1.2	Lloyd's Algorithm	14
6.1.3	K-means Extensions	14
6.2	Hierarchical Agglomerative Clustering (HAC)	14
6.2.1	HAC Algorithm	15
6.2.2	Linkage Criteria	15
6.2.3	Dendrogram	15
6.3	Comparing K-means and HAC	15
7	Mixture Models and Topic Models	16
7.1	Expectation-Maximization (EM) Algorithm	16
7.2	Mixture Models	16
7.2.1	Gaussian Mixture Model (GMM)	16
7.3	Directed Graphical Models (DGMs)	17
7.3.1	Expressing Models as DGMs	17
8	Dimensionality Reduction	17
8.1	Principal Component Analysis (PCA)	17
8.1.1	Variance Preservation View	18
8.1.2	Reconstruction Loss View	18
8.1.3	SVD and PCA	18
8.2	Applications of PCA	18
9	Graphical Models and Bayes Nets	18
9.1	Bayesian Networks	18
9.1.1	Conditional Probability Tables (CPTs)	19
9.1.2	Plate Notation	19
9.2	D-Separation and Independence	19
9.3	Variable Elimination	19
10	Hidden Markov Models	19
10.1	HMM Structure	19
10.2	Inference in HMMs	20
10.3	Forward-Backward Algorithm	20
10.4	Viterbi Algorithm	21
10.5	Learning in HMMs	21

11 Markov Decision Processes	21
11.1 MDP Framework	21
11.2 Value Functions	21
11.3 Bellman Equations	22
11.4 Planning in MDPs	22
11.4.1 Finite Horizon Planning	22
11.4.2 Value Iteration	22
11.4.3 Policy Iteration	23
12 Reinforcement Learning	23
12.1 Reinforcement Learning vs. Planning	23
12.2 Exploration vs. Exploitation	23
12.3 Model-Free vs. Model-Based RL	23
12.3.1 Model-Based RL	23
12.3.2 Model-Free RL	24
12.4 Q-Learning	24
12.5 SARSA	24
12.5.1 Differences Between Q-Learning and SARSA	24
13 Conclusion	24

1 Introduction

This study guide provides a comprehensive overview of key machine learning concepts covered in the course. For each topic, we provide definitions, mathematical formulations, intuitions, and examples to help solidify understanding.

2 Regression

2.1 Linear Regression Basics

Linear regression aims to model the relationship between a scalar dependent variable y and one or more explanatory variables x by fitting a linear equation to observed data.

2.1.1 The Bias Trick

Definition 1 (Bias Trick). *The bias trick is a technique where we append a constant feature $x_0 = 1$ to all of our data points, which allows us to incorporate the bias term w_0 into our weight vector, simplifying our equations.*

With the bias trick, we can rewrite:

$$f(x, w) = w_0 + w_1x_1 + \dots + w_Dx_D \quad (1)$$

$$= w^T x \quad (2)$$

where x now includes $x_0 = 1$ and w includes the bias term w_0 .

2.1.2 Least Squares Loss Function

Definition 2 (Least Squares Loss). *The least squares loss function measures the squared difference between the actual values and the predicted values:*

$$L(w) = \frac{1}{2} \sum_{n=1}^N (y_n - w^T x_n)^2 \quad (3)$$

The factor of $\frac{1}{2}$ is included to make the derivative cleaner.

2.1.3 Derivation of Optimal Weights

To find the optimal weights w^* that minimize the least squares loss function, we take the derivative of the loss with respect to w , set it to zero, and solve:

$$\nabla L(w) = \sum_{n=1}^N (y_n - w^T x_n)(-x_n) = 0 \quad (4)$$

$$\Rightarrow \sum_{n=1}^N y_n x_n - \sum_{n=1}^N (w^T x_n) x_n = 0 \quad (5)$$

In matrix form, with design matrix X (where each row is a data point x_n^T) and target vector y :

$$X^T y - X^T X w = 0 \quad (6)$$

$$\Rightarrow w^* = (X^T X)^{-1} X^T y \quad (7)$$

This is known as the normal equation for linear regression.

2.2 Basis Functions

Definition 3 (Basis Function). *A basis function $\phi(x)$ is a transformation applied to the input data to change its representation before applying linear regression. This allows linear models to capture non-linear relationships in the original feature space.*

The transformed linear regression model becomes:

$$y(x, w) = w^T \phi(x) \quad (8)$$

Common basis functions include:

- Polynomial: $\phi(x) = (1, x, x^2, x^3, \dots)$
- Radial Basis Functions (RBF): $\phi(x) = \exp\left(-\frac{\|x-\mu\|^2}{2\sigma^2}\right)$
- Sigmoid: $\phi(x) = \frac{1}{1+\exp(-x)}$

Basis functions allow us to fit more complex patterns while still using the machinery of linear regression.

2.3 Regularization

Definition 4 (Regularization). *Regularization is a technique used to prevent overfitting by adding a penalty term to the loss function that discourages large parameter values.*

Regularization is used when:

- The model is too complex and might overfit
- The data is noisy
- The features are correlated (multicollinearity)
- We want to enforce sparsity in the solution

2.3.1 Ridge Regression (L2 Regularization)

The loss function with L2 regularization:

$$L(w) = \frac{1}{2} \sum_{n=1}^N (y_n - w^T x_n)^2 + \frac{\lambda}{2} \|w\|_2^2 \quad (9)$$

The closed-form solution for ridge regression is:

$$w_{ridge}^* = (X^T X + \lambda I)^{-1} X^T y \quad (10)$$

Ridge regression prevents any individual weight from becoming too large, providing a solution with moderate values.

2.3.2 Lasso Regression (L1 Regularization)

The loss function with L1 regularization:

$$L(w) = \frac{1}{2} \sum_{n=1}^N (y_n - w^T x_n)^2 + \lambda \|w\|_1 \quad (11)$$

Lasso regression tends to produce sparse solutions (many weights set to exactly zero), effectively performing feature selection.

2.3.3 Elastic Net Regularization

Elastic Net combines L1 and L2 regularization:

$$L(w) = \frac{1}{2} \sum_{n=1}^N (y_n - w^T x_n)^2 + \lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2 \quad (12)$$

It balances the sparsity-inducing property of L1 with the stability of L2 regularization.

2.4 Bias-Variance Tradeoff

Definition 5 (Bias-Variance Tradeoff). *The bias-variance tradeoff describes the balance between two sources of error that prevent supervised learning algorithms from generalizing beyond their training set:*

- *High bias: Model is too simple and underfits the data*
- *High variance: Model is too complex and overfits the data*

The mean squared error of a model can be decomposed as:

$$\text{MSE} = \text{noise} + \text{bias}^2 + \text{variance} \quad (13)$$

- Adding more features or using more complex basis functions reduces bias but increases variance.
- Increasing regularization increases bias but reduces variance.
- Increasing training set size typically reduces variance without increasing bias.

2.5 Bayesian Linear Regression

In Bayesian linear regression, we place a prior distribution on the weights w :

$$p(w) = \mathcal{N}(w|0, S_0^{-1}) \quad (14)$$

The likelihood of the data is:

$$p(y|X, w, \beta) = \mathcal{N}(y|Xw, \beta^{-1}I) \quad (15)$$

By Bayes' rule, the posterior distribution of weights is:

$$p(w|X, y, \beta) \propto p(y|X, w, \beta)p(w) \quad (16)$$

With a normal prior and normal likelihood, the posterior is also normal:

$$p(w|X, y, \beta) = \mathcal{N}(w|\mu_N, S_N^{-1}) \quad (17)$$

where:

$$S_N = S_0 + \beta X^T X \quad (18)$$

$$\mu_N = S_N^{-1}(S_0 \mu_0 + \beta X^T y) \quad (19)$$

The connection to regularization: the prior in Bayesian linear regression is equivalent to adding regularization. A zero-mean Gaussian prior corresponds to L2 regularization, with the prior precision controlling the regularization strength.

2.5.1 Posterior Predictive Distribution

The posterior predictive distribution gives us uncertainty in our predictions for new inputs x_* :

$$p(y_*|x_*, X, y) = \int p(y_*|x_*, w)p(w|X, y)dw = \mathcal{N}(y_*|x_*^T \mu_N, \sigma_N^2(x_*)) \quad (20)$$

where $\sigma_N^2(x_*) = \beta^{-1} + x_*^T S_N^{-1} x_*$.

2.5.2 Marginal Likelihood

The marginal likelihood (evidence) is:

$$p(y|X) = \int p(y|X, w)p(w)dw \quad (21)$$

This measures how well a model explains the observed data, marginalizing over all possible parameter values.

3 Classification

3.1 Loss Functions for Classification

Classification aims to predict discrete class labels rather than continuous values. Several loss functions are used for classification:

3.1.1 0/1 Loss

Definition 6 (0/1 Loss). *The 0/1 loss assigns a loss of 1 for misclassification and 0 for correct classification.*

While intuitive, 0/1 loss is non-differentiable and thus difficult to optimize.

3.1.2 Hinge Loss

Definition 7 (Hinge Loss). *The hinge loss is defined as:*

$$L(w) = \sum_{i=1}^N \max(0, 1 - y_i(w^T x_i)) \quad (22)$$

where $y_i \in \{-1, 1\}$.

Hinge loss penalizes not only misclassifications but also correct classifications with low confidence (margin). It is convex and thus easier to optimize than 0/1 loss.

3.1.3 Logistic Loss

Definition 8 (Logistic Loss). *The logistic loss (cross-entropy loss) for binary classification is:*

$$L(w) = - \sum_{i=1}^N \{y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)\} \quad (23)$$

where $\hat{y}_i = \sigma(w^T x_i)$ and σ is the sigmoid function.

3.2 Gradient Descent

Definition 9 (Gradient Descent). *Gradient descent is an iterative optimization algorithm for finding the minimum of a function by moving in the direction of steepest descent.*

The update rule for gradient descent is:

$$w^{(t+1)} = w^{(t)} - \eta \nabla L(w^{(t)}) \quad (24)$$

where $\eta > 0$ is the learning rate.

Gradient descent works intuitively by:

- Computing the direction of steepest increase in loss (the gradient)
- Moving in the opposite direction to decrease loss
- Taking steps proportional to the learning rate η

If the learning rate is too high, the algorithm might overshoot the minimum and diverge. If it's too low, convergence will be very slow.

3.2.1 Batch vs. Stochastic Gradient Descent

- **Batch Gradient Descent:** Computes the gradient using the entire dataset at each step.
- **Stochastic Gradient Descent (SGD):** Computes the gradient using a single randomly selected data point at each step.
- **Mini-batch Gradient Descent:** Computes the gradient using a small random subset of the data at each step.

SGD has several advantages:

- Faster updates (especially with large datasets)
- Can escape local minima due to the noise in the gradient estimates
- Can be used for online learning

3.3 Perceptron Algorithm

The perceptron is a linear binary classifier that updates weights when misclassifications occur.

The perceptron update rule for a misclassified point (x_i, y_i) is:

$$w^{(t+1)} = w^{(t)} + \eta x_i y_i \quad (25)$$

Property 1. *If the data is linearly separable, the perceptron algorithm is guaranteed to converge to a solution that perfectly separates the classes in a finite number of steps.*

3.4 Logistic Regression

Definition 10 (Logistic Regression). *Logistic regression is a probabilistic discriminative model that estimates the probability of a binary class using the logistic sigmoid function:*

$$p(y = 1|x; w) = \sigma(w^T x) = \frac{1}{1 + \exp(-w^T x)} \quad (26)$$

The decision boundary of logistic regression is linear in the feature space, defined by the equation $w^T x = 0$. For binary classification, the log-likelihood is:

$$\log p(\{y_i\}_{i=1}^N | w) = \sum_{i=1}^N \{y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)\} \quad (27)$$

The gradient of the negative log-likelihood is:

$$\nabla E(w) = \sum_{i=1}^N (\hat{y}_i - y_i) x_i \quad (28)$$

3.4.1 Multi-class Logistic Regression

For multi-class problems, we use the softmax function:

$$p(y = k|x; W) = \frac{\exp(w_k^T x)}{\sum_{j=1}^K \exp(w_j^T x)} \quad (29)$$

The cross-entropy loss becomes:

$$E(W) = - \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log \hat{y}_{ik} \quad (30)$$

The gradient with respect to w_j is:

$$\nabla_{w_j} E(W) = \sum_{i=1}^N (\hat{y}_{ij} - y_{ij}) x_i \quad (31)$$

3.5 Generative vs. Discriminative Models

Definition 11 (Discriminative Model). *A discriminative model learns the conditional probability $p(y|x)$ directly, focusing on finding the decision boundary between classes.*

Definition 12 (Generative Model). *A generative model learns the joint probability $p(x, y) = p(x|y)p(y)$, modeling how the data was generated from each class.*

3.5.1 Generative Models for Classification

In generative models for classification:

- We model the class prior $p(y)$
- We model the class-conditional distribution $p(x|y)$
- We use Bayes' rule to get $p(y|x) = \frac{p(x|y)p(y)}{p(x)}$

3.5.2 Naive Bayes

Definition 13 (Naive Bayes). *Naive Bayes assumes that features are conditionally independent given the class:*

$$p(x|y) = \prod_{d=1}^D p(x_d|y) \quad (32)$$

Despite this strong independence assumption, Naive Bayes often works well in practice. It is particularly effective when:

- The dimensionality is high
- Training data is limited
- Features are indeed relatively independent

3.5.3 Comparing Discriminative and Generative Models

- Logistic Regression (discriminative): Models $p(y|x)$ directly
- Naive Bayes (generative): Models $p(x|y)$ and $p(y)$ to get $p(y|x)$

Generative models can:

- Generate new samples from the data distribution
- Incorporate prior knowledge about the data generation process
- Work well with missing data
- Often converge faster with less data

Discriminative models:

- Often achieve better accuracy when training data is plentiful
- Don't waste resources modeling aspects of the data irrelevant to the classification boundary
- Can be more robust to model misspecification

4 Neural Networks & Model Selection

4.1 Neural Network Basics

Definition 14 (Neural Network). *A neural network is a computational model inspired by the human brain, consisting of interconnected nodes (neurons) organized in layers. Each neuron applies a non-linear transformation to a weighted sum of its inputs.*

4.1.1 Activation Functions

Definition 15 (Activation Function). *An activation function introduces non-linearity into neural networks, allowing them to learn complex patterns.*

Activation functions are essential because:

- Without them, a multi-layer network would collapse to a single-layer linear model
- They allow the network to approximate arbitrary functions
- They determine the range of output values and affect gradient flow

Common activation functions include:

- Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$
- Hyperbolic Tangent (tanh): $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- Rectified Linear Unit (ReLU): $\text{ReLU}(z) = \max(0, z)$

4.1.2 Forward Propagation

For a feed-forward neural network with L layers:

$$z^{(1)} = W^{(1)}x + b^{(1)} \quad (33)$$

$$a^{(1)} = h(z^{(1)}) \quad (34)$$

$$\vdots \quad (35)$$

$$z^{(L)} = W^{(L)}a^{(L-1)} + b^{(L)} \quad (36)$$

$$\hat{y} = \sigma(z^{(L)}) \quad (37)$$

where h is the activation function, $W^{(l)}$ is the weight matrix for layer l , and $b^{(l)}$ is the bias vector.

4.2 Backpropagation

Definition 16 (Backpropagation). *Backpropagation is an algorithm for efficiently computing gradients in neural networks by propagating the error backward through the network.*

Backpropagation is useful because:

- Computing gradients directly would be computationally inefficient
- It leverages the chain rule to reuse computed quantities
- It scales well to deep networks

The backpropagation algorithm works as follows:

1. Perform a forward pass to compute activations for all layers
2. Compute the error at the output layer

3. Propagate the error backward through the network, computing gradients for each parameter
4. Update the parameters using the computed gradients

For the output layer L , the error is:

$$\delta^{(L)} = \frac{\partial E}{\partial z^{(L)}} = \hat{y} - y \quad (38)$$

For hidden layers l , the error is:

$$\delta^{(l)} = \frac{\partial E}{\partial z^{(l)}} = ((W^{(l+1)})^T \delta^{(l+1)}) \odot h'(z^{(l)}) \quad (39)$$

where $\odot$ is element-wise multiplication.

The gradients for weights and biases are:

$$\frac{\partial E}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T \quad (40)$$

$$\frac{\partial E}{\partial b^{(l)}} = \delta^{(l)} \quad (41)$$

4.3 Neural Networks for Regression and Classification

Neural networks can be used for both regression and classification tasks by choosing appropriate:

- Output layer architecture
- Activation functions
- Loss functions

4.3.1 Regression with Neural Networks

For regression:

- Output layer: Linear activation (identity function)
- Loss function: Mean Squared Error (MSE)

$$E = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (42)$$

4.3.2 Classification with Neural Networks

For binary classification:

- Output layer: One neuron with sigmoid activation
- Loss function: Binary Cross-Entropy

$$E = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] \quad (43)$$

For multi-class classification:

- Output layer: K neurons with softmax activation
- Loss function: Categorical Cross-Entropy

$$E = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log \hat{y}_{ik} \quad (44)$$

4.4 Model Selection and Regularization

Definition 17 (Model Selection). *Model selection is the process of choosing the best model architecture and hyperparameters for a given task.*

4.4.1 Cross-Validation

Cross-validation helps prevent overfitting by:

- Dividing the data into training and validation sets
- Training the model on the training set
- Evaluating it on the validation set
- Selecting the model with the best validation performance

K-fold cross-validation divides the data into K subsets (folds), training on K-1 folds and validating on the remaining fold, repeating K times.

4.4.2 Regularization in Neural Networks

Regularization techniques for neural networks include:

- L1/L2 regularization: Adding penalty terms to the loss function based on the norm of the weights

$$E_{reg} = E + \lambda \sum_{l=1}^L \sum_{i,j} (W_{ij}^{(l)})^2 \quad (45)$$

- Dropout: Randomly setting a fraction of neurons to zero during training
- Early stopping: Stopping training when validation error starts to increase
- Data augmentation: Creating additional training data through transformations

5 Support Vector Machines (SVMs)

5.1 Maximum Margin Classification

Definition 18 (Maximum Margin Classifier). *A maximum margin classifier finds the decision boundary that maximizes the distance between the boundary and the closest data points from each class.*

Support Vector Machines (SVMs) are maximum margin classifiers that aim to find the hyperplane that maximizes this margin.

5.2 Hard-Margin SVM

For linearly separable data, the hard-margin SVM optimization problem is:

$$\min_{w, w_0} \frac{1}{2} \|w\|^2 \quad (46)$$

$$\text{s.t. } y_i(w^T x_i + w_0) \geq 1, \text{ for all } i \quad (47)$$

The margin is given by $\frac{1}{\|w\|}$, so minimizing $\|w\|$ maximizes the margin.

5.3 Soft-Margin SVM

For non-linearly separable data, the soft-margin SVM introduces slack variables $\xi_i \geq 0$:

$$\min_{w, w_0, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \quad (48)$$

$$\text{s.t. } y_i(w^T x_i + w_0) \geq 1 - \xi_i, \text{ for all } i \quad (49)$$

$$\xi_i \geq 0, \text{ for all } i \quad (50)$$

The regularization parameter C controls the trade-off between maximizing the margin and minimizing the classification error:

- Large C : Focus on minimizing violations (low bias, high variance)
- Small C : Focus on maximizing margin (high bias, low variance)

5.4 Hinge Loss in SVMs

The soft-margin SVM optimization is equivalent to minimizing:

$$\frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \max(0, 1 - y_i(w^T x_i + w_0)) \quad (51)$$

The second term is the hinge loss, which penalizes:

- Misclassified points ($y_i(w^T x_i + w_0) < 0$)
- Correctly classified points that are too close to the boundary ($0 < y_i(w^T x_i + w_0) < 1$)

5.5 Kernel Trick

Definition 19 (Kernel Trick). *The kernel trick allows SVMs to operate in a high-dimensional feature space without explicitly computing the transformation of the input data.*

Instead of computing $\phi(x_i)^T \phi(x_j)$ directly, we use a kernel function $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$.

Common kernel functions include:

- Linear: $K(x_i, x_j) = x_i^T x_j$
- Polynomial: $K(x_i, x_j) = (x_i^T x_j + 1)^d$
- Radial Basis Function (RBF): $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$

The kernel trick is useful because:

- It enables efficient computation in potentially infinite-dimensional spaces
- It allows non-linear decision boundaries in the original feature space
- It can be applied to various algorithms beyond SVMs

5.6 Support Vectors

Definition 20 (Support Vectors). *Support vectors are the data points that lie on the margin boundaries or violate the margin. They are the only points that influence the position of the decision boundary.*

Support vectors are important because:

- They determine the decision boundary (removing other points doesn't change it)
- They are typically a small subset of the training data
- They allow for efficient prediction on new data

5.7 SVM vs Logistic Regression

Both SVM and logistic regression produce linear decision boundaries but differ in how they:

- Measure error: SVMs use hinge loss, logistic regression uses logistic loss
- Handle outliers: SVMs can be more robust due to the maximum margin
- Provide probabilities: Logistic regression naturally gives probabilities, SVMs don't
- Scale with dataset size: SVMs can be more efficient for medium-sized datasets

6 Clustering

6.1 K-means Clustering

Definition 21 (K-means Clustering). *K-means clustering aims to partition N observations into K clusters, where each observation belongs to the cluster with the nearest mean.*

6.1.1 K-means Objective

The K-means objective is to minimize the within-cluster sum of squares:

$$J = \sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2 \quad (52)$$

where C_k is the set of points in cluster k and μ_k is the mean of cluster k .

6.1.2 Lloyd's Algorithm

Lloyd's algorithm is the standard approach for K-means clustering:

Algorithm 1 Lloyd's Algorithm for K-means

- 1: Initialize cluster centers $\mu_1, \mu_2, \dots, \mu_K$
 - 2: **repeat**
 - 3: Assign each point to the nearest cluster center: $r_{nk} = \begin{cases} 1 & \text{if } k = \arg \min_{k'} \|x_n - \mu_{k'}\|^2 \\ 0 & \text{otherwise} \end{cases}$
 - 4: Update cluster centers: $\mu_k = \frac{\sum_{n=1}^N r_{nk} x_n}{\sum_{n=1}^N r_{nk}}$
 - 5: **until** convergence
-

Lloyd's algorithm is guaranteed to converge to a local minimum of the objective function.

6.1.3 K-means Extensions

Extensions to K-means include:

- K-means++: A smarter initialization that improves convergence and final clustering quality
- K-medoids: Uses actual data points as cluster centers, more robust to outliers

6.2 Hierarchical Agglomerative Clustering (HAC)

Definition 22 (Hierarchical Agglomerative Clustering). *HAC builds a hierarchy of clusters by starting with each data point as its own cluster and successively merging the closest pairs of clusters.*

Algorithm 2 Hierarchical Agglomerative Clustering

- 1: Start with N clusters, one for each data point
 - 2: **repeat**
 - 3: Find the two closest clusters using the chosen linkage criterion
 - 4: Merge these two clusters
 - 5: Update distances between the new cluster and all other clusters
 - 6: **until** only one cluster remains
-

6.2.1 HAC Algorithm

6.2.2 Linkage Criteria

Different linkage criteria define how the distance between clusters is measured:

- Single linkage (min-linkage): Minimum distance between any two points in different clusters

$$d(C_i, C_j) = \min_{x \in C_i, y \in C_j} \|x - y\| \quad (53)$$

- Complete linkage (max-linkage): Maximum distance between any two points in different clusters

$$d(C_i, C_j) = \max_{x \in C_i, y \in C_j} \|x - y\| \quad (54)$$

- Average linkage: Average distance between all pairs of points in different clusters

$$d(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i} \sum_{y \in C_j} \|x - y\| \quad (55)$$

- Centroid linkage: Distance between cluster centroids

$$d(C_i, C_j) = \|\mu_i - \mu_j\| \quad (56)$$

The choice of linkage criterion affects the shape of the resulting clusters:

- Single linkage: Tends to produce elongated, "stringy" clusters
- Complete linkage: Tends to produce compact, equal-diameter clusters
- Average linkage: Strikes a balance between the two extremes

6.2.3 Dendrogram

A dendrogram is a tree diagram that visualizes the hierarchy of clusters in HAC. The height at which two clusters merge represents the distance between them.

6.3 Comparing K-means and HAC

- K-means:
 - Requires specifying the number of clusters K in advance
 - Produces flat partitioning (no hierarchy)
 - Generally faster than HAC ($O(NKD)$ per iteration)
 - Result depends on initialization
 - Tends to find spherical clusters
- HAC:

- Does not require specifying the number of clusters in advance
- Produces a hierarchy of clusterings
- Slower than K-means ($O(N^2 \log N)$)
- Deterministic (no initialization needed)
- Can find clusters of various shapes depending on linkage

7 Mixture Models and Topic Models

7.1 Expectation-Maximization (EM) Algorithm

Definition 23 (Expectation-Maximization). *The Expectation-Maximization (EM) algorithm is an iterative method for finding maximum likelihood estimates of parameters in statistical models with latent variables.*

The EM algorithm alternates between:

- E-step: Compute the expected value of the latent variables given the current parameters
- M-step: Update the parameters to maximize the expected complete-data log-likelihood

The Evidence Lower Bound (ELBO) is a lower bound on the log-likelihood:

$$\log p(X|\theta) \geq \mathcal{L}(q, \theta) = \mathbb{E}_{q(Z)}[\log p(X, Z|\theta)] - \mathbb{E}_{q(Z)}[\log q(Z)] \quad (57)$$

where $q(Z)$ is a distribution over the latent variables.

The EM algorithm:

- E-step: Find $q^{(t+1)} = \arg \max_q \mathcal{L}(q, \theta^{(t)})$, which gives $q^{(t+1)}(Z) = p(Z|X, \theta^{(t)})$
- M-step: Find $\theta^{(t+1)} = \arg \max_{\theta} \mathcal{L}(q^{(t+1)}, \theta)$

7.2 Mixture Models

Definition 24 (Mixture Model). *A mixture model is a probabilistic model that assumes the data is generated from a mixture of several probability distributions.*

The general mixture model assumes each data point x_i is generated as follows:

- Sample a component $z_i \sim \text{Categorical}(\pi)$
- Sample a data point $x_i \sim p(x|z_i, \theta)$

The marginal distribution is:

$$p(x|\pi, \theta) = \sum_{k=1}^K \pi_k p(x|z = k, \theta_k) \quad (58)$$

7.2.1 Gaussian Mixture Model (GMM)

In a Gaussian Mixture Model, each component distribution is a Gaussian:

$$p(x|\pi, \mu, \Sigma) = \sum_{k=1}^K \pi_k \mathcal{N}(x|\mu_k, \Sigma_k) \quad (59)$$

EM for GMM:

- E-step: Compute responsibilities

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)} \quad (60)$$

- M-step: Update parameters

$$\pi_k^{new} = \frac{1}{N} \sum_{i=1}^N \gamma_{ik} \quad (61)$$

$$\mu_k^{new} = \frac{\sum_{i=1}^N \gamma_{ik} x_i}{\sum_{i=1}^N \gamma_{ik}} \quad (62)$$

$$\Sigma_k^{new} = \frac{\sum_{i=1}^N \gamma_{ik} (x_i - \mu_k^{new})(x_i - \mu_k^{new})^T}{\sum_{i=1}^N \gamma_{ik}} \quad (63)$$

7.3 Directed Graphical Models (DGMs)

Definition 25 (Directed Graphical Model). *A directed graphical model (DGM) is a way to represent the conditional independence structure in a probability distribution using a directed graph.*

In a DGM:

- Nodes represent random variables or parameters
- Directed edges represent conditional dependencies
- Shaded nodes represent observed variables
- Unshaded nodes represent latent variables or parameters
- Plates represent repeated structures

7.3.1 Expressing Models as DGMs

For example, a Gaussian Mixture Model can be represented as:

- A node for the mixture weights π
- A plate containing N data points, each with:
 - A latent variable z_i (unshaded)
 - An observed variable x_i (shaded)
- Arrows from π to z_i and from z_i to x_i

8 Dimensionality Reduction

8.1 Principal Component Analysis (PCA)

Definition 26 (Principal Component Analysis). *PCA is a dimensionality reduction technique that finds a low-dimensional representation of the data that captures as much of the variance as possible.*

8.1.1 Variance Preservation View

PCA finds the directions (principal components) along which the data varies the most. Mathematically, the first principal component w_1 maximizes:

$$w_1 = \arg \max_{||w||=1} \frac{1}{N} \sum_{i=1}^N (w^T x_i)^2 = \arg \max_{||w||=1} w^T S w \quad (64)$$

where $S = \frac{1}{N} X^T X$ is the empirical covariance matrix.

The solution is the eigenvector of S with the largest eigenvalue. Subsequent principal components are the eigenvectors with decreasingly smaller eigenvalues, subject to being orthogonal to all previous principal components.

8.1.2 Reconstruction Loss View

PCA can also be viewed as finding a low-dimensional representation that minimizes the reconstruction error when projecting back to the original space:

$$\min_W \sum_{i=1}^N ||x_i - WW^T x_i||^2 \quad (65)$$

where W is a matrix with orthonormal columns.

Both views lead to the same solution: the columns of W are the principal components (eigenvectors of S).

8.1.3 SVD and PCA

PCA is closely related to the Singular Value Decomposition (SVD) of the data matrix X :

$$X = U \Sigma V^T \quad (66)$$

The principal components are the columns of V , and the singular values in Σ are related to the eigenvalues of the covariance matrix by a factor of $\sqrt{N}$.

8.2 Applications of PCA

PCA is used for:

- Visualization: Reducing data to 2 or 3 dimensions for plotting
- Noise reduction: Discarding dimensions with low variance
- Feature extraction: Using the principal components as new features
- Data compression: Storing a low-dimensional representation

9 Graphical Models and Bayes Nets

9.1 Bayesian Networks

Definition 27 (Bayesian Network). *A Bayesian Network (BN) is a directed acyclic graph (DAG) where nodes represent random variables and edges represent direct dependencies.*

In a Bayesian Network:

- Each node X_i is conditionally independent of its non-descendants given its parents

- The joint probability factorizes as:

$$p(X_1, X_2, \dots, X_n) = \prod_{i=1}^n p(X_i | pa(X_i)) \quad (67)$$

where $pa(X_i)$ are the parents of X_i

9.1.1 Conditional Probability Tables (CPTs)

For discrete variables, the conditional distributions $p(X_i | pa(X_i))$ are represented using Conditional Probability Tables (CPTs).

9.1.2 Plate Notation

Plate notation is used to represent repeated structures in graphical models. A plate (rectangle) around nodes indicates that those nodes are replicated, with the number of replications shown in the corner.

9.2 D-Separation and Independence

Definition 28 (D-Separation). *D-separation is a criterion for determining whether sets of variables are conditionally independent given a Bayesian network structure.*

Two variables X and Y are d-separated by a set of variables Z if and only if all paths between X and Y are blocked by Z .

A path is blocked if it contains:

- A chain $A \rightarrow B \rightarrow C$ where B is in Z
- A fork $A \leftarrow B \rightarrow C$ where B is in Z
- A collider $A \rightarrow B \leftarrow C$ where B and all its descendants are not in Z

If X and Y are d-separated by Z , then X and Y are conditionally independent given Z .

9.3 Variable Elimination

Definition 29 (Variable Elimination). *Variable elimination is an algorithm for exact inference in Bayesian networks by eliminating variables one by one from the joint distribution.*

For example, to compute $p(X_1 | X_2 = x_2)$:

$$p(X_1 | X_2 = x_2) = \frac{p(X_1, X_2 = x_2)}{p(X_2 = x_2)} \quad (68)$$

$$= \frac{\sum_{X_3, \dots, X_n} p(X_1, X_2 = x_2, X_3, \dots, X_n)}{p(X_2 = x_2)} \quad (69)$$

The efficiency of variable elimination depends on the elimination order. A good heuristic is to eliminate leaf nodes first, as this typically results in smaller intermediate factors.

10 Hidden Markov Models

10.1 HMM Structure

Definition 30 (Hidden Markov Model). *A Hidden Markov Model (HMM) is a statistical model where the system is assumed to be a Markov process with unobservable (hidden) states and observable emissions.*

An HMM is characterized by:

- A set of hidden states $S = \{s_1, s_2, \dots, s_K\}$
- A set of possible emissions $E = \{e_1, e_2, \dots, e_M\}$
- Initial state distribution π where $\pi_i = p(s_1 = i)$
- Transition probabilities A where $A_{ij} = p(s_{t+1} = j | s_t = i)$
- Emission probabilities B where $B_{ik} = p(x_t = k | s_t = i)$

The joint probability of a sequence of states and emissions is:

$$p(s_1, \dots, s_n, x_1, \dots, x_n) = p(s_1) \prod_{t=1}^{n-1} p(s_{t+1} | s_t) \prod_{t=1}^n p(x_t | s_t) \quad (70)$$

10.2 Inference in HMMs

Common inference questions in HMMs include:

- Filtering: $p(s_t | x_1, \dots, x_t)$ - What is the current state given all observations up to now?
- Prediction: $p(s_{t+k} | x_1, \dots, x_t)$ - What will be the state in the future?
- Smoothing: $p(s_t | x_1, \dots, x_n)$ - What was the state at time t given all observations?
- Most likely path: $\arg \max_{s_1, \dots, s_n} p(s_1, \dots, s_n | x_1, \dots, x_n)$ - What is the most likely sequence of states?

10.3 Forward-Backward Algorithm

The Forward-Backward algorithm computes:

- Forward probabilities $\alpha_t(i) = p(x_1, \dots, x_t, s_t = i)$
- Backward probabilities $\beta_t(i) = p(x_{t+1}, \dots, x_n | s_t = i)$

Forward recursion:

$$\alpha_1(i) = \pi_i B_{i, x_1} \quad (71)$$

$$\alpha_{t+1}(j) = B_{j, x_{t+1}} \sum_{i=1}^K \alpha_t(i) A_{ij} \quad (72)$$

Backward recursion:

$$\beta_n(i) = 1 \quad (73)$$

$$\beta_t(i) = \sum_{j=1}^K A_{ij} B_{j, x_{t+1}} \beta_{t+1}(j) \quad (74)$$

These can be used to compute:

$$p(s_t = i | x_1, \dots, x_n) = \frac{\alpha_t(i) \beta_t(i)}{p(x_1, \dots, x_n)} \quad (75)$$

$$p(s_t = i, s_{t+1} = j | x_1, \dots, x_n) = \frac{\alpha_t(i) A_{ij} B_{j, x_{t+1}} \beta_{t+1}(j)}{p(x_1, \dots, x_n)} \quad (76)$$

10.4 Viterbi Algorithm

The Viterbi algorithm finds the most likely sequence of hidden states:

$$\arg \max_{s_1, \dots, s_n} p(s_1, \dots, s_n | x_1, \dots, x_n) \quad (77)$$

It uses dynamic programming to compute:

$$\delta_1(i) = \pi_i B_{i,x_1} \quad (78)$$

$$\delta_t(j) = \max_i [\delta_{t-1}(i) A_{ij}] B_{j,x_t} \quad (79)$$

and keeps track of the argmax at each step to reconstruct the optimal path.

10.5 Learning in HMMs

The parameters of an HMM can be learned using the EM algorithm (Baum-Welch algorithm):

- E-step: Compute expected counts of transitions and emissions using the Forward-Backward algorithm
- M-step: Update parameters to maximize the expected complete-data log-likelihood

11 Markov Decision Processes

11.1 MDP Framework

Definition 31 (Markov Decision Process). *A Markov Decision Process (MDP) is a mathematical framework for modeling decision-making in situations where outcomes are partly random and partly under the control of a decision-maker.*

An MDP is defined by:

- A set of states S
- A set of actions A
- Transition probabilities $P(s'|s, a)$ - the probability of transitioning to state s' when taking action a in state s
- Reward function $R(s, a)$ - the immediate reward received after taking action a in state s
- Discount factor $\gamma \in [0, 1]$ - how much future rewards are valued compared to immediate rewards

11.2 Value Functions

Definition 32 (Value Function). *The value function $V^\pi(s)$ is the expected return starting from state s and following policy π .*

For a policy π , the value function is:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right] \quad (80)$$

Definition 33 (Q-Function). *The Q-function $Q^\pi(s, a)$ is the expected return starting from state s , taking action a , and then following policy π .*

The Q-function is:

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \quad (81)$$

The optimal value function $V^*(s)$ is the maximum value achievable from state s :

$$V^*(s) = \max_{\pi} V^\pi(s) \quad (82)$$

The optimal Q-function $Q^*(s, a)$ is:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \quad (83)$$

11.3 Bellman Equations

The Bellman equations are recursive relationships for value functions:

Definition 34 (Bellman Consistency Equation). *For a given policy π :*

$$V^\pi(s) = \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right] \quad (84)$$

Definition 35 (Bellman Optimality Equation). *For the optimal policy:*

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right] \quad (85)$$

These equations form the basis for planning and learning algorithms in MDPs.

11.4 Planning in MDPs

11.4.1 Finite Horizon Planning

For a finite time horizon T , we compute:

$$V_t(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_{t+1}(s') \right] \quad (86)$$

starting from $V_T(s) = \max_a R(s, a)$ and working backwards.

11.4.2 Value Iteration

Value iteration is an algorithm for infinite horizon planning:

Algorithm 3 Value Iteration

- 1: Initialize $V(s) = 0$ for all $s \in S$
 - 2: **repeat**
 - 3: $\Delta \leftarrow 0$
 - 4: **for** each $s \in S$ **do**
 - 5: $v \leftarrow V(s)$
 - 6: $V(s) \leftarrow \max_a [R(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s')]$
 - 7: $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 - 8: **end for**
 - 9: **until** $\Delta < \theta$ (a small threshold)
-

Value iteration converges to the optimal value function V^* .

Algorithm 4 Policy Iteration

- 1: Initialize π arbitrarily
 - 2: **repeat**
 - 3: **Policy Evaluation:** Compute $V^\pi(s)$ for all $s \in S$
 - 4: **Policy Improvement:** Update $\pi(s) \leftarrow \arg \max_a [R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s')]$
 - 5: **until** policy stable
-

11.4.3 Policy Iteration

Policy iteration alternates between policy evaluation and policy improvement:

Policy iteration converges in fewer iterations than value iteration but may require more computation per iteration.

12 Reinforcement Learning

12.1 Reinforcement Learning vs. Planning

Definition 36 (Reinforcement Learning). *Reinforcement learning is a type of machine learning where an agent learns to make decisions by interacting with an environment and receiving feedback in the form of rewards.*

The key difference between RL and planning is that in RL, the agent:

- Does not know the transition probabilities $P(s'|s, a)$
- Does not know the reward function $R(s, a)$
- Must learn from experience through trial and error

12.2 Exploration vs. Exploitation

A fundamental challenge in RL is the exploration-exploitation tradeoff:

- Exploration: Trying new actions to discover their effects
- Exploitation: Using known information to maximize rewards

Strategies for balancing exploration and exploitation include:

- ϵ -greedy: Take the best known action with probability $1 - \epsilon$, and a random action with probability ϵ
- Softmax: Choose actions with probability proportional to their estimated values
- Upper Confidence Bound (UCB): Choose actions based on both their estimated value and the uncertainty in that estimate

12.3 Model-Free vs. Model-Based RL

12.3.1 Model-Based RL

In model-based RL, the agent:

- Learns a model of the environment (transition probabilities and rewards)
- Uses the model for planning (e.g., with value iteration or policy iteration)

12.3.2 Model-Free RL

In model-free RL, the agent:

- Learns value functions or policies directly from experience
- Does not explicitly model the environment

12.4 Q-Learning

Definition 37 (Q-Learning). *Q-Learning is a model-free RL algorithm that learns the optimal Q-function directly from experience.*

The Q-learning update rule is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \quad (87)$$

where α is the learning rate.

Q-learning is an off-policy algorithm because it learns about the optimal policy regardless of the policy being followed.

12.5 SARSA

Definition 38 (SARSA). *SARSA (State-Action-Reward-State-Action) is a model-free RL algorithm that learns the Q-function for the policy being followed.*

The SARSA update rule is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \quad (88)$$

SARSA is an on-policy algorithm because it learns about the policy being followed.

12.5.1 Differences Between Q-Learning and SARSA

The key differences between Q-learning and SARSA are:

- Q-learning uses the maximum Q-value for the next state ($\max_{a'} Q(s_{t+1}, a')$)
- SARSA uses the Q-value of the actual next action taken ($Q(s_{t+1}, a_{t+1})$)
- Q-learning converges to the optimal policy regardless of the exploration strategy
- SARSA converges to the optimal policy only if exploration is gradually reduced
- SARSA can be more conservative in risky environments

13 Conclusion

This comprehensive study guide has covered the key concepts and techniques in machine learning, from basic regression and classification to advanced topics like reinforcement learning. Understanding these fundamentals provides a solid foundation for applying machine learning to real-world problems and exploring more advanced topics in the field.